BCSE307L COMPILER DESIGN · PROJECT A24 · TEAM 8 · REVIEW 3

BURS-Based Retargetable Instruction Selection

One IR, one bottom-up rewrite engine, two instruction sets: cost-optimal code, verified by simulation

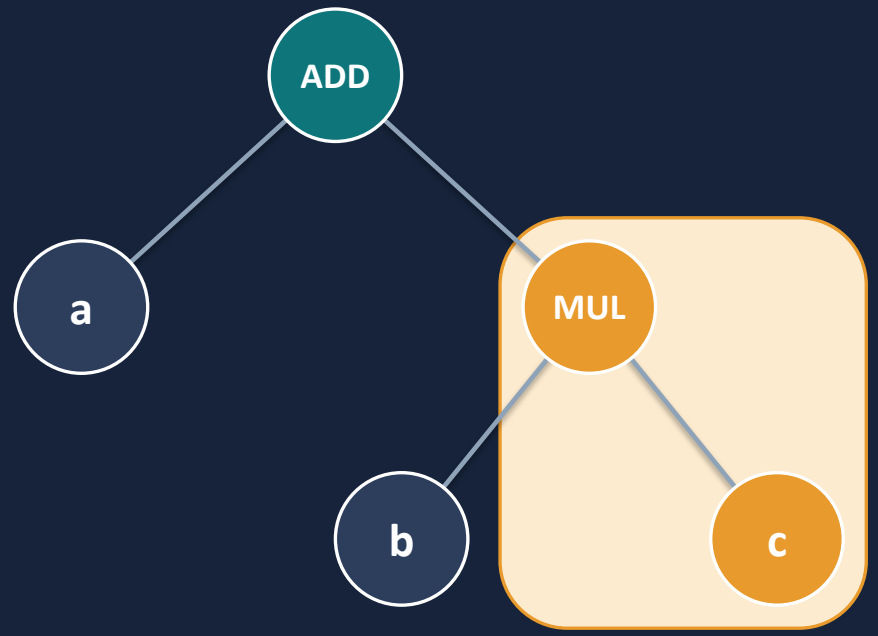
Vineet Kumar Mishra
Project lead · IR & architecture

Mayank Verma
Target grammars & costs

Utsav Keshrwani
BURS matching engine

Ahmad Azim
Emission, testing & release

Faculty: Dr. Shalini L · github.com/ahmadazim542/burs-instruction-selector




```
PUSH [a]; PUSH [b]; MULM [c]; ADD
```

a + b * c on Target B: one MULM covers two tree nodes

Instruction selection is a covering problem

Presenter **Vineet**

The same IR tree can be compiled many ways; the cheapest choice depends on the target machine

Example: load(a) + 4 on Target A

NAIVE: cost 4

```
LOAD  r0,[a]
LOAD  r1,[r0]
LOADI r2,#4
ADD   r3,r1,r2
```

BURS: cost 3

```
LOAD  r0,[a]
LOAD  r1,[r0]
ADDI  r2,r1,#4
```

Why it matters

Optimality: choices interact across subtrees, so greedy per-node choices miss cheaper code

Retargeting: each processor has different instructions and costs; hand-written selectors do not port

Teaching value: a complete, inspectable back-end technique in under 2,000 lines

Measurable objectives: 7 / 7 achieved

- ✓ Expression-tree IR: CONST, VAR, ADD, SUB, MUL, DIV, LOAD
- ✓ Target patterns, templates and costs (11 + 17 rules)
- ✓ Bottom-up labeling with DP cost computation
- ✓ Minimum-cost cover selected when rules compete
- ✓ Backtrack the cover and emit target instructions
- ✓ Two ISAs from one target-independent IR and engine
- ✓ Reproducible tests and evidence: 33 / 33 pass

Review 2 actions are closed

Presenter **Vineet**

Everything pending at Review 2 is done; scope as in Review 1, plus a DIV operator

0% → 100%

Target B: rule table, emitter, simulator

6 → 33

automated checks (was: 6 hand-checked expressions)

Public

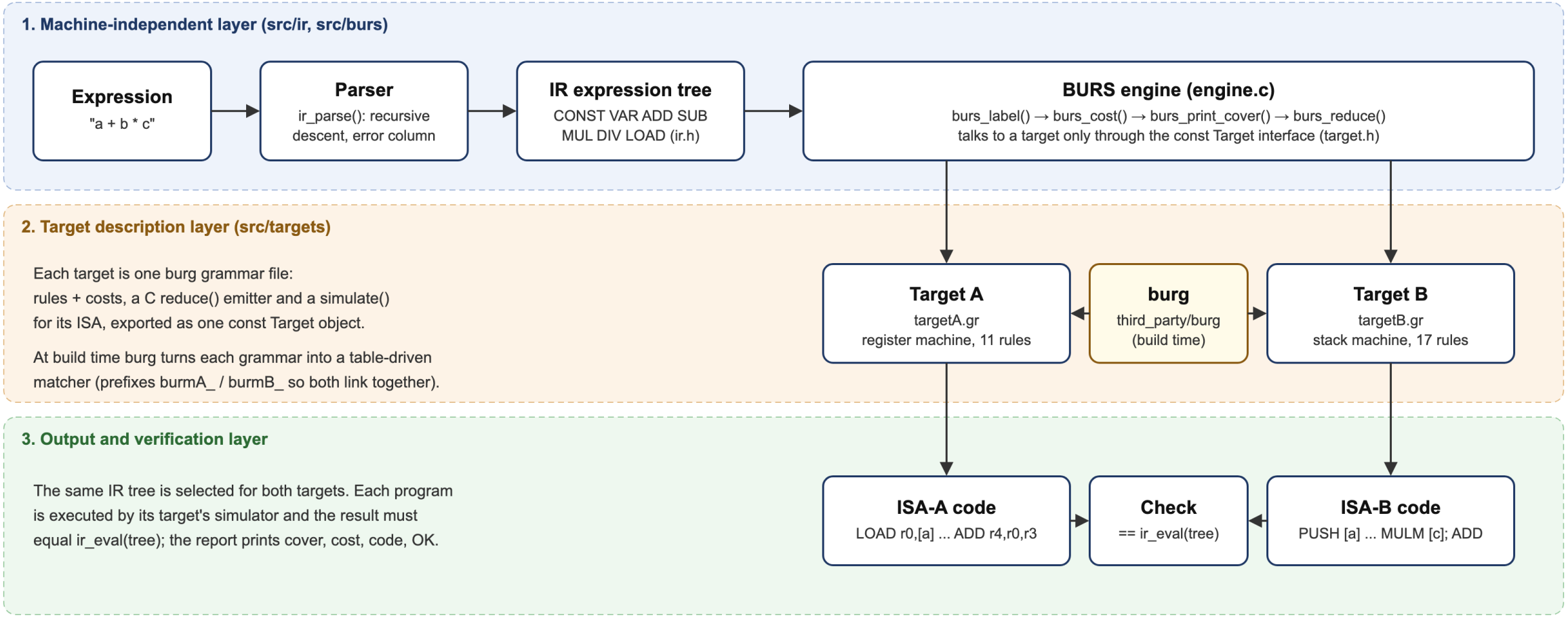
GitHub repository with code, tests, results, docs

Review 2 item	Action taken	Status
Target-B rule table not defined	17-rule zero-address stack ISA with memory and immediate operands (incl. DIV)	Closed
D3: Target-B emitter missing	reduceB() emitter and simulateB() checker	Closed
Run test expressions on Target B	All review expressions run on both targets	Closed
Expand regression suite	31 stored cases + 11-expression suite + 7,140-tree exhaustive check	Closed
Team GitHub repository	github.com/ahmadazim542/burs-instruction-selector	Closed
Register numbers on cover	Utsav and Ahmad added; Vineet and Mayank pending	Pending

Architecture: only the grammar knows the machine

Presenter **Vineet**

Common core (src/ir, src/burs) → one grammar file per target → verified output for both



Compiler concepts and design decisions

Four ideas from the course, and the decisions that shaped the implementation

Presenter **Vineet · Mayank · Utsav**

1 Tree grammar

nt: PATTERN = rule (cost)

one rule ≈ one instruction pattern

2 Nonterminals + chain rules

reg, stk, mem, imm say where a value lives; stk: mem converts

3 Bottom-up DP

best(node, nt) = min over matching rules of cost + children's best

4 BURG tables

DP precomputed at build time: one lookup per node, O(n)

Design decisions

Precomputed BURG tables

over run-time DP: linear labeling, optimality guaranteed by the generator

One const Target interface

the engine has no target code; a new ISA = one grammar + one line

Simulate every program

an independent oracle: catches wrong covers and wrong emitters

Module 1: two target descriptions

Presenter **Mayank**

Same %term line: CONST=1 VAR=2 ADD=3 SUB=4 MUL=5 LOAD=6 DIV=7; different rules, nonterminals and costs

TARGET A · targetA.gr · register machine · 11 rules

```
reg: VAR           = 1 (1);   LOAD  rd,[var]
reg: CONST         = 2 (1);   LOADI rd,#k
imm: CONST         = 3 (0);   operand #k
reg: LOAD(reg)     = 4 (1);   LOAD  rd,[rs]
reg: ADD(reg,reg)  = 5 (1);   ADD
reg: ADD(reg,imm)  = 6 (1);   ADDI
reg: SUB(reg,reg)  = 7 (1);   SUB
reg: SUB(reg,imm)  = 8 (1);   SUBI
reg: MUL(reg,reg)  = 9 (2);   MUL
reg: DIV(reg,reg)  = 10 (3);  DIV
reg: DIV(reg,imm)  = 11 (3);  DIVI
```

TARGET B · targetB.gr · stack machine · 17 rules

```
stk: mem           = 1 (1);    PUSH  [var]
stk: imm           = 2 (1);    PUSHI #k
mem: VAR           = 3 (0);    operand
imm: CONST         = 4 (0);    operand
stk: LOAD(stk)     = 5 (1);    LOAD
stk: ADD(stk,stk)  = 6 (1);    ADD
stk: ADD(stk,mem)  = 7 (1);    ADDM  [var]
stk: ADD(stk,imm)  = 8 (1);    ADDI  #k
stk: SUB(stk, stk|mem|imm) 9-11
stk: MUL(stk,stk)  = 12 (2);   MUL
stk: MUL(stk,mem)  = 13 (2);   MULM  [var]
stk: MUL(stk,imm)  = 14 (2);   MULI  #k
stk: DIV(stk, stk|mem|imm) 15-17 (3)
```

Amber: operand forms and chain rules. They let one node offer several "shapes" to its parent, which is where the cost savings come from.

Module 1 demo: the cheapest rule wins

Presenter **Mayank**

Live commands and their real output

```
./build/burs -t A "a + 7"
```

cheapest rule chosen

```
LOAD r0,[a]
ADDI r1,r0,#7
cost 2  (LOADI + ADD would be 3)
```

```
./build/burs -t B "a * b"
```

cheapest rule chosen

```
PUSH [a]
MULM [b]
cost 3  (PUSH,PUSH,MUL would be 4)
```

```
./build/burs -t A "7 + a"
```

known limitation

```
LOADI r0,#7
LOAD r1,[a]
ADD r2,r0,r1
cost 3: no commutative rule
```

```
./build/burs "a % b"
```

invalid input rejected

```
ERROR: invalid expression,
column 3: operator '%' is not
supported. exit status 2
```

Module 2: the BURS engine

Presenter **Utsav**

engine.c: four target-independent functions; every call goes through the Target object

1

`burs_label`

postorder walk; one table lookup per node writes its state

2

`burs_cost`

best rule + cost for every node and nonterminal

3

`burs_print_cover`

backtrack from (root, start NT): the selected rules

4

`burs_reduce`

children first, then target.reduce(): emits code

src/burs/engine.c (excerpt)

```
Operand burs_reduce(const Target *t, NODEPTR_TYPE p,
                    int nt, EmitCtx *ctx)
{
  int r = t->rule(STATE_LABEL(p), nt);
  short *nts = t->nts[r];
  ...
  t->kids(p, r, kids);
  /* children first, left to right: */
  /* instructions come out in postorder */
  for (i = 0; nts[i]; i++)
    ops[i] = burs_reduce(t, kids[i], nts[i], ctx);
  return t->reduce(ctx, p, r, ops);
}
```

Module 2 demo: the cost table behind a choice

Presenter **Utsav**

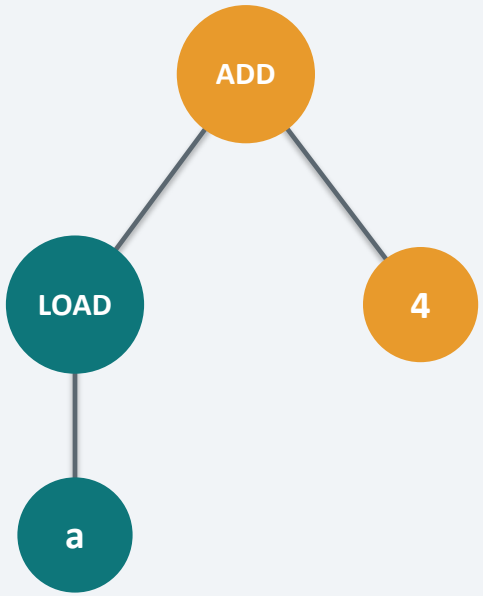
./build/burs -v -t A "load(a) + 4": DP result per node (n1..n4 numbered in postorder)

Node	NT	Best rule	Cost	Why
n1 VAR a	reg	reg: VAR	1	only rule
n2 LOAD	reg	reg: LOAD(reg)	2	1 + cost(n1, reg)
n3 CONST 4	reg	reg: CONST	1	LOADI
n3 CONST 4	imm	imm: CONST	0	folds into parent
n4 ADD	reg	reg: ADD(reg,imm)	3	1 + 2 + 0 = 3 < ADD(reg,reg) = 4

GENERATED CODE

```
LOAD r0,[a]
LOAD r1,[r0]
ADDI r2,r1,#4
Total cover cost : 3    Simulation : 1006 = 1006  OK
```

The selected cover



Amber: ADDI covers ADD and the constant together. Teal: one LOAD each.

End-to-end: one tree, two machines, both verified

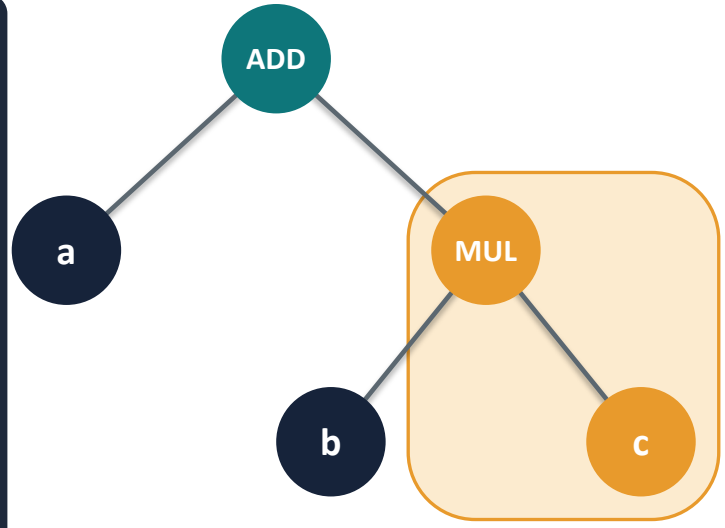
Presenter **All · Ahmad drives**

./build/burs "a + b * c": test values a=2, b=3, c=4; reference value 14

TARGET A · REGISTER MACHINE

```
LOAD r0,[a]
LOAD r1,[b]
LOAD r2,[c]
MUL  r3,r1,r2
ADD  r4,r0,r3

cost 6 · 5 instr · sim 14 OK
```



amber tile = Target B's MULM

TARGET B · STACK MACHINE

```
PUSH [a]
PUSH [b]
MULM [c]
ADD


cost 5 · 4 instr · sim 14 OK
```

Demo script

burs = ./build/burs, run from the repository root

1. `make clean && make` — three build stages
2. `burs "a + b * c"` — normal case
3. `burs "[[a] + 1]"` — boundary
4. `burs "(a + b"` — invalid input
5. `burs -i` — evaluator's input

Test strategy: every output is checked twice

Presenter **Ahmad**

Oracle 1: byte-exact expected output · Oracle 2: simulate the generated code and compare with the IR value

33 / 33

checks pass (make test)

13,692

exhaustive runs, 0 failures

7,140

trees of depth ≤ 2, incl. DIV

Positive — 8

review expressions and division on both targets: tree, cover, cost, code

Boundary — 7

single leaf, deep nesting, nested loads, named vars, big constants, -7 / 2

Rule choice — 7

ADDI, SUBI, MULM, MULI, DIVI, DIVM chosen; 7 + a: no commutative rule

Negative + failure — 8 + 1

%, missing operand, unbalanced ([, unary minus, …; a / (b - b) exit 3

Regression — 11

--suite: all review expressions, both targets, PASS/FAIL table

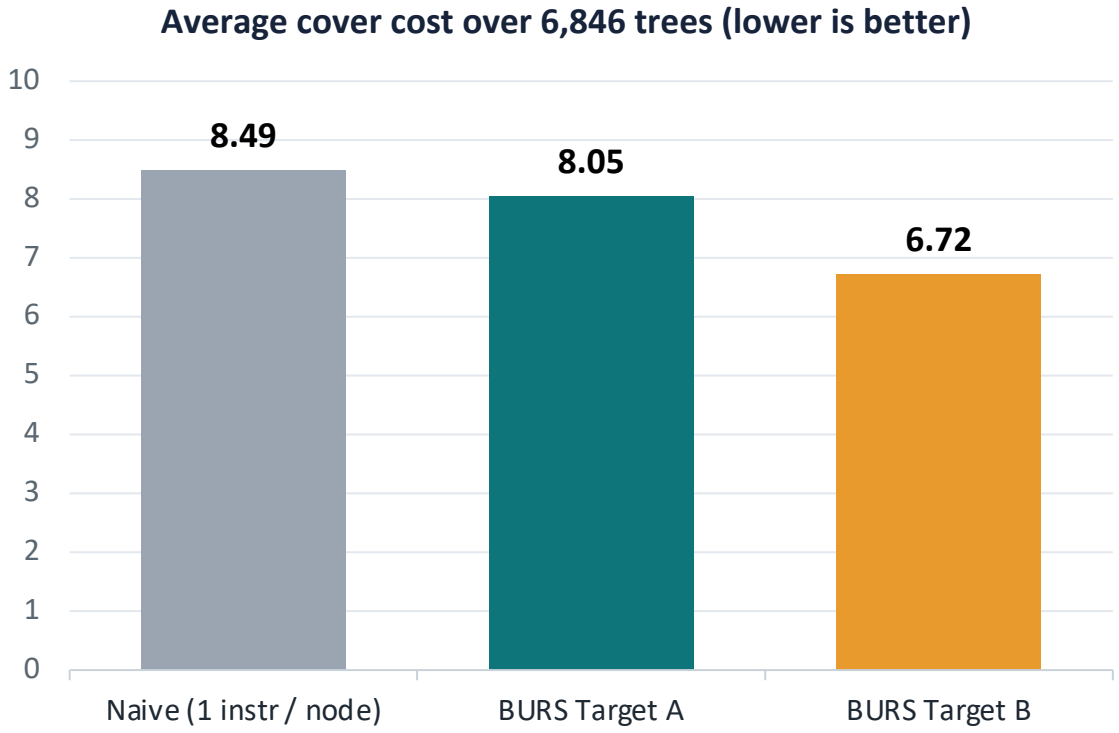
Exhaustive — 7,140

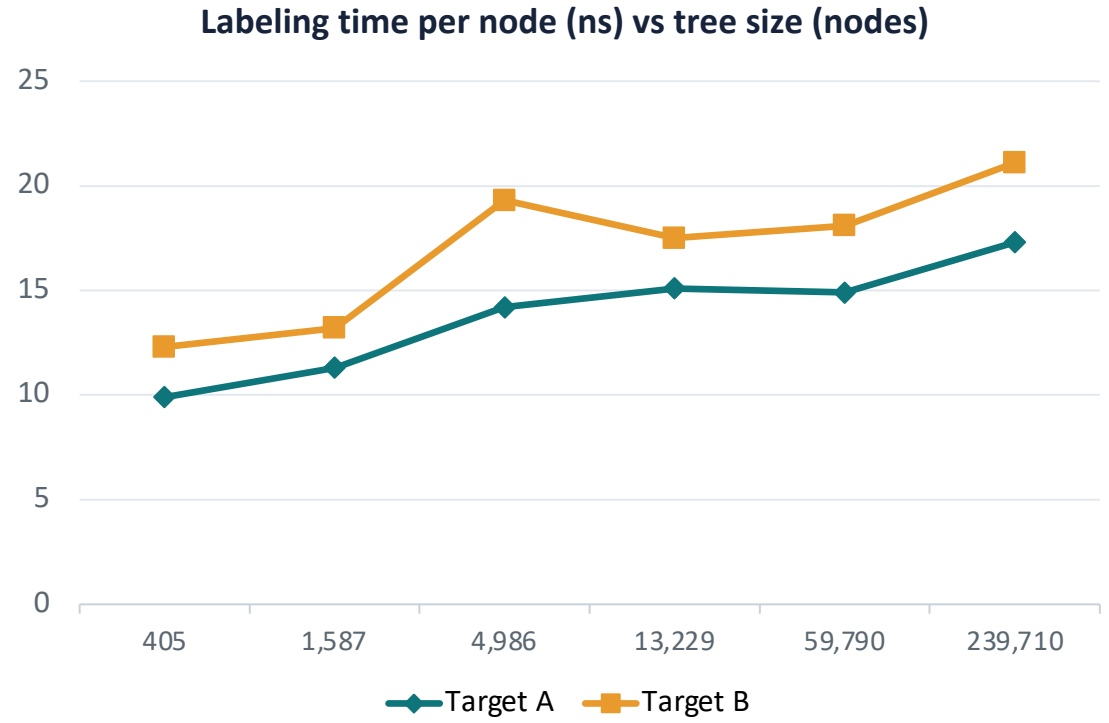
--exhaustive: every small tree incl. DIV, both targets, plus baseline

Results: correct, cheaper and linear-time

Presenter **Ahmad**

From results/exhaustive_check.txt and results/benchmark.txt (Apple M1)





13,692 / 13,692

generated programs correct

−20.9%

cost on Target B vs naive

−5.1%

cost on Target A vs naive

O(n)

≈9–21 ns per node up to 240k nodes

Limitations, failure cases and trade-offs

What the system does not do, what that costs, and how it would be fixed

Presenter **Utsav · Ahmad**

Trees, not DAGs

Impact repeated subexpressions are recomputed

Remedy split DAGs into trees before selection

Static rule costs

Impact cannot say "small immediates are cheaper"

Remedy iburg-style dynamic costs

No register allocation

Impact 39 virtual registers for the deep test

Remedy Sethi–Ullman or linear scan

No commutative rules

Impact 7 + a: cost 3 instead of 2

Remedy add reg: ADD(imm,reg), stk: ADD(mem,stk)

Limited operators

Impact %, unary minus rejected (exit 2)

Remedy new IR op + rules per target (as done for DIV)

Recursion depth

Impact extremely deep unbalanced trees hit the C stack

Remedy iterative traversal

Repository and reproducibility

Presenter **Ahmad**

github.com/ahmadazim542/burs-instruction-selector · C99 + make · no other dependencies

REPOSITORY LAYOUT

```
burs-instruction-selector/
├── Makefile          3-stage build
├── src/
│   ├── ir/           parser, IR, ir_eval
│   ├── burs/         engine + Target API
│   ├── targets/
│   │   ├── targetA/  targetA.gr
│   │   └── targetB/  targetB.gr
│   └── main/         driver, modes
├── tests/            31 cases + expected/
├── results/          committed evidence
├── docs/             Review 1–3 documents
└── third_party/burg   BURG generator
```

1. **`git clone …/burs-instruction-selector.git`**
   source, tests, results, docs

2. **`make`**
   build burg → generate both matchers → build/burs

3. **`make test`**
   33 passed, 0 failed

4. **`make results`**
   regenerate results/*.txt and compare

Member-wise technical contribution

Presenter **All**

Ownership approved in Review 1; each member owns files, tests and a viva topic

VK

Vineet Kumar Mishra

Project lead · IR & front end

OWNS

src/ir/ir.h, ir.c
parser, IR contract, ir_eval
build pipeline

EVIDENCE

neg_* tests

VIVA

architecture, IR, parser

MV

Mayank Verma

Target descriptions

OWNS

targetA.gr: 11 rules
targetB.gr: 17 rules
nonterminals, costs

EVIDENCE

rule_* tests

VIVA

grammars, chain rules

UK

Utsav Keshrwani

BURS engine

OWNS

src/burs/engine.c
target.h interface
burg prefixes, cost table

EVIDENCE

-v tables, --bench

VIVA

DP, BURG tables

AA

Ahmad Azim

Emission, testing, release

OWNS

reduceA/B, simulateA/B
main.c, tests/, results/
README, GitHub repo

EVIDENCE

33/33, --exhaustive

VIVA

emission, verification

Beyond the minimum, and what comes next

Presenter **Vineet**

What we added beyond the approved scope, and where it can go

Beyond the brief

★ **Two targets in one binary**

burg -p prefixes: the same tree, compared side by side

★ **Simulation oracle**

every generated program is executed and checked

★ **Exhaustive check**

7,140 trees incl. DIV × 2 targets, 0 failures

★ **Baseline + benchmark**

quantified savings and linear scaling

Next steps

Register allocation for Target A (Sethi–Ullman / linear scan)

Third target: an RV32I (RISC-V) subset with addressing modes

DAGs and statements; commutative and dynamic-cost rules

Visualiser that draws the selected cover on the tree

Publication potential

An educational case study or lab module: one matching engine, two ISAs, automatic verification, quantitative comparison.

CONCLUSION

Change the grammar, not the compiler

1 Optimal by construction

Tree covering is dynamic programming; BURG precomputes it, so selection is O(n).

2 Nonterminals are locations

reg, stk, mem, imm and chain rules give each node several shapes; that is the saving.

3 Interface + oracle

One Target API and a simulator per ISA make a new target cheap and trustworthy.

7 / 7
objectives achieved

33 / 33
checks pass

−20.9%
cost on Target B

2 ISAs
one engine

Thank you. Questions, or an expression of your choice: ./build/burs -i

References and acknowledgements

Presenter **Ahmad**

Sources consulted, reused tools, and how they were used

[1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., 2006, Ch. 8.

[2] A. V. Aho, M. Ganapathi, S. W. K. Tjiang, "Code Generation Using Tree Matching and Dynamic Programming," ACM TOPLAS 11(4), 1989.

[3] C. W. Fraser, D. R. Hanson, T. A. Proebsting, "Engineering a Simple, Efficient Code-Generator Generator," ACM LOPLAS 1(3), 1992.

[4] C. W. Fraser, R. R. Henry, T. A. Proebsting, "BURG: Fast Optimal Instruction Selection and Tree Parsing," SIGPLAN Notices 27(4), 1992.

[5] T. A. Proebsting, "Simple and Efficient BURS Table Generation," PLDI 1992.

[6] Chen Zhexuan, burg (modernised BURG), github.com/CloseToZero/burg.

Acknowledgements

BURG generator. third_party/burg, unmodified, used as a build tool under its own licence (COPYRIGHT, LICENSE.TXT). Not counted as team code.

AI assistance. Implementation and documentation were assisted by an AI coding assistant (Claude, Anthropic), as recorded in the commits and the report. The team reviewed and tested all submitted work.

Course. BCSE307L lecture material on back ends and instruction selection; Dr. Shalini L for guidance.

Backup: emitter and simulator code

Presenter **Ahmad**

targetA.gr reduceA() · targetB.gr simulateB() (excerpts)

src/targets/targetA/targetA.gr · reduceA()

```
switch (rule) {
case 3:   /* imm: CONST -> operand only */
  return burs_operand("#%lld", p->ival);
case 1:   /* reg: VAR */
  d = burs_new_reg(ctx);
  burs_emit(ctx, "LOAD %s,[%s]", d.s, p->name);
  return d;
case 5: case 6: ... case 11:  /* ADD..DIVI */
  d = burs_new_reg(ctx);
  burs_emit(ctx, "%s %s,%s,%s",
            mnem[rule], d.s, k[0].s, k[1].s);
  return d;
}
```

src/targets/targetB/targetB.gr · simulateB()

```
/* ADD..DIV: pop b, pop a, push a op b */
if (sp < 2) return simfail(..., "stack underflow");
if (!arith(op[0], S[sp-2], S[sp-1], &S[sp-2]))
  return simfail(..., "division by zero");
sp--;
/* ADDM/ADDI etc.: top op operand */
if (!operand_value(a[0], &v) ||
    (op[3] == 'M') != (a[0][0] == '['))
  return simfail(..., "bad operand");
if (!arith(op[0], S[sp-1], v, &S[sp-1])) ...
/* end: depth must be exactly 1 */
*value = S[0];   /* compared with ir_eval() */
```

Backup: how a regression test runs

Presenter **Ahmad**

tests/cases.txt → tests/run_tests.sh → tests/expected/<id>.out

tests/cases.txt (excerpt)

```
# id                  | options | expression
pos_a_plus_b_mul_c    | -v      | a + b * c
rule_divm             | -t B    | a / b
neg_modulo            |         | a % b
```

tests/run_tests.sh (core loop)

```
$BURS $opts "$expr" >"$TMP" 2>&1
echo "exit=$?" >>"$TMP"

if cmp -s "$TMP" "$EXPECTED/$id.out"; then
  pass=$((pass + 1))
else
  fail=$((fail + 1)); diff ... | head -20
fi
```

1. **Run**
   build/burs with the case's options and expression
2. **Capture**
   stdout + stderr + exit status into one file
3. **Compare**
   cmp byte-for-byte with tests/expected/<id>.out
4. **Then**
   --suite (11/11) and --exhaustive (0 failures)
5. **Result**
   33 passed, 0 failed; non-zero exit on any failure